

WinDbg / cdb Workflow Guide for OSED

This guide is ordered the way OSED work usually unfolds: start the debug session, triage the crash, inspect stack and memory, then move into module enumeration, PE walking, and exploit-development helpers. The syntax is the same debugger syntax you would use in WinDbg or cdb, but the examples are chosen for x86 exploit work first. Default number base is **hex**. Use 0n100 when you mean decimal 100. Registers are written as **@eax**, **@esp**, and **@eip**. **poi(x)** means “read the pointer stored at x”. WinDbg comments use **\$\$** inside scripts.

1. Start the session

Command	Why use it	Example
<code>cdb -o -G target.exe args</code>	Launch under the debugger and keep child processes visible.	<code>cdb -o -G vuln_strcpy_x86.exe AAAA</code>
<code>cdb -cf script.wds target.exe</code>	Run a command script at the initial loader break, then continue.	<code>cdb -cf setup.wds vuln_strcpy_x86.exe</code>
<code>windbgx -o -g target.exe args</code>	Same idea in WinDbg Preview. Use this when you want the GUI.	<code>windbgx -o -g vuln_strcpy_x86.exe AAAA</code>
<code>windbg -p <pid></code> <code>windbg -pn name.exe</code> <code>.sympath</code> <code>srv*c:\sym*https://msdl.microsoft.com/download/symbols</code> <code>.reload /f</code>	Attach to a running process by PID. Attach by image name when the PID is not stable. Set a symbol path that can pull Microsoft symbols on demand. Force symbol reload after you change the symbol path or load a new module.	<code>windbg -p 4128</code> <code>windbg -pn vuln_strcpy_x86.exe</code> <code>.sympath</code> <code>srv*c:\sym*https://msdl.microsoft.com/download/symbols</code> <code>.reload /f</code>
<code>lm</code> <code>lm m vuln*</code> <code>lmf</code> <code>x ntdll!*Ldr*</code>	List loaded modules. Good first check after attach. Filter the module list to a target pattern. Show loaded modules with their paths. Search symbols by wildcard when you know the subsystem but not the exact name.	<code>lm</code> <code>lm m vuln*</code> <code>lmf</code> <code>x ntdll!*Ldr*</code>
<code>ln 0x76f7db6b</code> <code>!analyze -v</code>	Ask WinDbg for the nearest symbol to an address. Let WinDbg summarize the exception and common crash clues.	<code>ln 0x76f7db6b</code> <code>!analyze -v</code>
<code>dx @\$osed().help()</code>	If the osed-windbg toolkit is loaded, show command help first.	<code>dx @\$osed().help()</code>
<code>dx @\$osed().triage()</code>	Run a fast read-only crash triage for control, SEH, stack, and modules.	<code>dx @\$osed().triage()</code>

Threads and expressions

Use this section when the debugger output is ambiguous and you need to confirm which thread owns the state you are reading, or when you need to do explicit address math instead of relying on a higher-level helper.

Command	Why use it	Example
<code>-</code>	List all threads, their TEBs, and current execution state.	<code>-</code>
<code>-!s</code>	Switch to thread 1 when you want a different execution context.	<code>-!s</code>
<code>~*k</code> <code>? expr</code>	Show the call stacks for all threads at once. Evaluate an expression, including pointer arithmetic and offsets.	<code>~*k</code> <code>? @esp+0x20</code>
<code>? poi(@rsp)</code>	Dereference a stack pointer and inspect the value it points to.	<code>? poi(@rsp)</code>
<code>? poi(@esp+4)</code> <code>? @\$ra</code>	Common pattern for following the first stack argument. Show the current return address without manually dereferencing the stack.	<code>? poi(@esp+4)</code> <code>? @\$ra</code>
<code>?? expr</code>	Use the alternate expression evaluator when you want C++-style evaluation.	<code>?? (void**)@rsp</code>
<code>.formats expr</code> <code>k 2</code>	Inspect a value in multiple numeric representations. Limit the stack trace to a small number of frames for quick triage.	<code>.formats poi(@esp)</code> <code>k 2</code>
<code>dx (void**)@rsp</code>	Typed pointer inspection bridge between raw memory and dx.	<code>dx (void**)@rsp</code>

2. Read the crash state

Command	Why use it	Example
<code>r</code>	Dump registers, flags, and the next instruction in one shot.	<code>r</code>
<code>r eip</code> <code>r esp</code>	Check whether instruction pointer control is obvious. Confirm the current stack pointer before reading stack data.	<code>r eip</code> <code>r esp</code>
<code>.exr -1</code> <code>.ecxr</code> <code>k</code> <code>kb</code>	Show the last exception record. Switch to the exception context register state. Show the current call stack quickly. Show the stack with the first three arguments for each frame.	<code>.exr -1</code> <code>.ecxr</code> <code>k</code> <code>kb</code>
<code>kp</code>	Show the stack with full parameters when symbols support it.	<code>kp</code>
<code>dps esp L40</code>	Dump stack pointers with symbol resolution. This is a common post-crash view.	<code>dps esp L40</code>
<code>dd esp L20</code>	Read raw DWORDs from the stack when you want exact values.	<code>dd esp L20</code>
<code>da poi(esp+4)</code>	Follow the argument pointer and read the pointed-to ASCII string.	<code>da poi(esp+4)</code>
<code>db esp L20</code>	Inspect the first bytes near the stack pointer.	<code>db esp L20</code>

3. Control execution

Command	Why use it	Example
<code>g</code>	Continue execution.	<code>g</code>
<code>g 0x401000</code>	Run until a specific address.	<code>g 0x401000</code>
<code>p</code>	Step over one instruction.	<code>p</code>
<code>t</code>	Step into one instruction.	<code>t</code>
<code>pa 0x401050</code>	Step over until an address is reached.	<code>pa 0x401050</code>
<code>pt</code>	Step to the next <code>ret</code> .	<code>pt</code>
<code>pc</code>	Step to the next <code>call</code> .	<code>pc</code>
<code>gu</code>	Run until the current function returns.	<code>gu</code>
<code>.restart</code>	Restart the target from scratch after a bad run.	<code>.restart</code>
<code>q</code>	Quit the debugger.	<code>q</code>
<code>qd</code>	Quit and detach.	<code>qd</code>

4. Place breakpoints

Command	Why use it	Example
<code>bp module+0x1821</code>	Break at a module-relative offset.	<code>bp vuln_strcpy_x86+0x1821</code>
<code>bp kernel32!WinExec</code>	Break on a known symbol.	<code>bp kernel32!WinExec</code>
<code>bu module!func</code>	Deferred breakpoint that resolves after the module loads.	<code>bu msvcrt!strcpy</code>
<code>ba w4 0x402000</code>	Break when 4 bytes at an address are written.	<code>ba w4 0x402000</code>
<code>bp address "command; g"</code>	Run a command list on break, then continue.	<code>bp kernel32!CreateFileA "kb; dd esp L10; g"</code>
<code>bl</code>	List active breakpoints.	<code>bl</code>
<code>bc *</code>	Clear all breakpoints.	<code>bc *</code>
<code>bd 0</code>	Disable breakpoint 0 without deleting it.	<code>bd 0</code>
<code>be 0</code>	Re-enable breakpoint 0.	<code>be 0</code>

5. Read registers and flags

Command	Why use it	Example
<code>r eax</code>	Show one register.	<code>r eax</code>
<code>r eax=0x41414141</code>	Set a register while testing assumptions.	<code>r eax=0x41414141</code>
<code>r \$t0=0x100</code>	Use a scratch pseudo-register for notes or temporary values.	<code>r \$t0=0x100</code>
<code>r @eip</code>	Read the instruction pointer without symbol lookup.	<code>r @eip</code>
<code>r output flags</code>	Read <code>zr</code> , <code>cy</code> , <code>ov</code> , <code>pl</code> , and <code>mi</code> when the flags matter.	<code>r</code>

6. Inspect memory

Command	Why use it	Example
<code>db addr L20</code>	View raw bytes and ASCII together.	<code>db esp L20</code>
<code>dd addr L40</code>	View DWORDs. This is the workhorse for stack reads.	<code>dd esp L40</code>
<code>dps addr L20</code>	View DWORDs and resolve any pointers to symbols.	<code>dps esp L20</code>
<code>dc addr</code>	View DWORDs as ASCII and hex.	<code>dc esp L10</code>
<code>dw addr</code>	View 16-bit words.	<code>dw esp L10</code>
<code>dq addr</code>	View 64-bit values. Useful on x64 or when reading 8-byte fields.	<code>dq rsp L10</code>
<code>da addr</code>	Read ASCII text at an address.	<code>da poi(esp+4)</code>
<code>du addr</code>	Read UTF-16 text at an address.	<code>du poi(esp+4)</code>
<code>dt ntdll!_PEB @\$peb</code>	Dump the current process environment block if symbols are present.	<code>dt ntdll!_PEB @\$peb</code>
<code>dt ntdll!_TEB @\$teb</code>	Dump the current thread environment block if symbols are present.	<code>dt ntdll!_TEB @\$teb</code>
<code>!address esp</code>	Check the memory region and protection flags around a pointer.	<code>!address esp</code>
<code>eb addr 90 90</code>	Patch bytes in memory.	<code>eb eip 90 90</code>
<code>ed addr 0x41414141</code>	Patch a DWORD in memory.	<code>ed esp 0x41414141</code>

7. Disassemble and inspect code

Command	Why use it	Example
<code>u eip</code>	Disassemble forward from the current instruction.	<code>u eip</code>
<code>u addr L20</code>	Disassemble a specific range.	<code>u 0x401000 L20</code>
<code>ub addr</code>	Disassemble backward to find the call that led here.	<code>ub 0x401050</code>
<code>uf module!func</code>	Disassemble a whole function.	<code>uf kernel32!WinExec</code>
<code>u poi(@esp)</code>	Disassemble the return target or saved code pointer on the stack.	<code>u poi(@esp)</code>
<code>dx @\$osed().sc.base("kernel32")</code>	Use the toolkit to resolve a module base before reading code around it.	<code>dx @\$osed().sc.base("kernel32")</code>

8. Follow the stack and frames

Command	Why use it	Example
<code>.frame N</code>	Switch to a different stack frame.	<code>.frame 2</code>
<code>dv</code>	Show local variables when private symbols are available.	<code>dv</code>
<code>k</code>	Re-check the stack after changing frame or context.	<code>k</code>
<code>kb</code>	Use when you want arguments alongside return addresses.	<code>kb</code>
<code>kp</code>	Use when you want fuller parameter decoding.	<code>kp</code>

Stripped binary rule of thumb: read the prologue. `sub esp,0x208` tells you how much local space was reserved. `[ebp-N]` is a local, `[ebp+8..]` are arguments, and `[ebp+4]` is the saved return address.

9. Search memory

Command	Why use it	Example
<code>s -a 0 L780000000 "TRUN"</code> <code>s -u 0 L780000000 "calc"</code> <code>s -b 0 L780000000 90 90 90 90</code> <code>s -d 0 L780000000 0x41414141</code> <code>dx @\$osed().pattern_create(300, "msf")</code> <code>dx @\$osed().pattern_offset(0x39654138, "msf")</code> <code>dx @\$osed().exploit("offset")</code>	Search for an ASCII string across memory. Search for a UTF-16 string. Search for a raw byte pattern. Search for a DWORD value. Generate a cyclic pattern for offset finding. Convert a crash value into an offset. Generate the usual pattern workflow commands in one step.	<code>s -a 0 L780000000 "TRUN"</code> <code>s -u 0 L780000000 "calc"</code> <code>s -b 0 L780000000 90 90 90 90</code> <code>s -d 0 L780000000 0x41414141</code> <code>dx @\$osed().pattern_create(300, "msf")</code> <code>dx @\$osed().pattern_offset(0x39654138, "msf")</code> <code>dx @\$osed().exploit("offset")</code>

10. Walk the process model and loaded modules

Command	Why use it	Example
<code>!teb</code> <code>!peb</code> <code>dt ntdll!_PEB @\$peb</code> <code>dt ntdll!_TEB @\$teb</code> <code>lm</code> <code>lm m kernel*</code> <code>!dh kernel32</code> <code>!lmi kernel32</code> <code>dx @\$osed().sc.peb()</code> <code>dx @\$osed().sc.modules()</code> <code>dx @\$osed().sc.modules("kernel")</code>	Show thread environment block details quickly. Show process environment block details quickly. Read the current PEB as a typed structure. Read the current TEB as a typed structure. Confirm which modules are loaded. Filter to the modules that matter. Inspect PE headers for a loaded module. Show detailed loaded-module information. Ask the toolkit to summarize PEB state. Ask the toolkit for a module table. Filter the toolkit module view.	<code>!teb</code> <code>!peb</code> <code>dt ntdll!_PEB @\$peb</code> <code>dt ntdll!_TEB @\$teb</code> <code>lm</code> <code>lm m kernel*</code> <code>!dh kernel32</code> <code>!lmi kernel32</code> <code>dx @\$osed().sc.peb()</code> <code>dx @\$osed().sc.modules()</code> <code>dx @\$osed().sc.modules("kernel")</code>

11. Read PE headers and exports

Command	Why use it	Example
<code>!dh -f module</code> <code>!dh module</code> <code>dx @\$osed().sc.exportdir("kernel32")</code> <code>dx @\$osed().sc.export("kernel32", "GetProcAddress")</code> <code>dx @\$osed().sc.exportwalk("kernel32", "GetProcAddress")</code> <code>dx @\$osed().sc.exportat("kernel32", 842)</code> <code>dx @\$osed().sc.hashes("kernel32", "crc32")</code> <code>dx @\$osed().sc.hash("WinExec", "ROR13")</code> <code>dx @\$osed().sc.hashresolve("kernel32", 0x7c0dfcaa, "ROR13")</code> <code>dx @\$osed().sc.algorithms()</code>	Dump headers and file layout for a module. Inspect the DOS and NT headers for the module. Show the export directory in a structured view. Resolve a named export directly. Walk exports the way shellcode would. Resolve by ordinal/index when that is all you have. See hash values for a whole module. Compute a hash for one API name. Reverse a hash to a likely API name. List the supported hash algorithms.	<code>!dh -f kernel32</code> <code>!dh kernel32</code> <code>dx @\$osed().sc.exportdir("kernel32")</code> <code>dx @\$osed().sc.export("kernel32", "GetProcAddress")</code> <code>dx @\$osed().sc.exportwalk("kernel32", "GetProcAddress")</code> <code>dx @\$osed().sc.exportat("kernel32", 842)</code> <code>dx @\$osed().sc.hashes("kernel32", "crc32")</code> <code>dx @\$osed().sc.hash("WinExec", "ROR13")</code> <code>dx @\$osed().sc.hashresolve("kernel32", 0x7c0dfcaa, "ROR13")</code> <code>dx @\$osed().sc.algorithms()</code>

12. Use exploit-development helpers

Command	Why use it	Example
<code>!exchain</code> <code>sxe av</code> <code>sxd av</code> <code>sxe ld:module</code> <code>.exr -1</code> <code>.ecxr</code> <code>dx @\$osed().seh()</code> <code>dx @\$osed().seh_ppr("libsp.dll", "00 0A 0D", 50, true, "fast")</code> <code>dx @\$osed().pivots("essfunc")</code> <code>dx @\$osed().rop("essfunc")</code> <code>dx @\$osed().rop_suggest("essfunc")</code> <code>dx @\$osed().find_bytes("essfunc", "FF E4")</code> <code>dx @\$osed().badchars(0x00B8F900)</code> <code>.writemem C:\dbg\dump.bin poi(@esp+4)+0x7d6</code> <code>(poi(@esp+4)+0x7d6)+0x200</code> <code>.logopen c:\dbg\log.txt</code> <code>.logclose</code> <code>.printf "eip=%x\n", @eip</code> <code>.echo BADCHAR_CRASH</code> <code>\$\$<c:\s.wds</code> <code>.foreach (a {!some}) { u a }</code> <code>.shell -ci "cmd" cmd /c move ...</code>	Inspect the SEH chain during exception work. Break on access violations so you can stop at the fault. Ignore access violations when you want the program to continue. Break when a module loads. Useful for deferred breakpoint work. Re-check the exception record when you return to the crash. Switch into the exception context before inspecting registers. Inspect the SEH chain with toolkit support. Find usable pop; pop; ret candidates for SEH work. Search for stack pivots in a chosen module. Scope the gadget search to a module. Get validated gadget suggestions. Search a module for a specific instruction byte sequence. Compare memory against an expected byte sequence after a badchar probe. Dump a candidate buffer for offline comparison. Capture output to a file for later review. Stop logging. Print a formatted line inside a script or command list. Emit a marker that is easy to spot in logs. Run a script file. Iterate over the output of another command. Run a host command from the debugger.	<code>!exchain</code> <code>sxe av</code> <code>sxd av</code> <code>sxe ld:kernel32</code> <code>.exr -1</code> <code>.ecxr</code> <code>dx @\$osed().seh()</code> <code>dx @\$osed().seh_ppr("libsp.dll", "00 0A 0D", 50, true, "fast")</code> <code>dx @\$osed().pivots("essfunc")</code> <code>dx @\$osed().rop("essfunc")</code> <code>dx @\$osed().rop_suggest("essfunc")</code> <code>dx @\$osed().find_bytes("essfunc", "FF E4")</code> <code>dx @\$osed().badchars(0x00B8F900)</code> <code>.writemem C:\dbg\dump.bin poi(@esp+4)+0x7d6</code> <code>(poi(@esp+4)+0x7d6)+0x200</code> <code>.logopen c:\dbg\log.txt</code> <code>.logclose</code> <code>.printf "eip=%x\n", @eip</code> <code>.echo BADCHAR_CRASH</code> <code>\$\$<c:\s.wds</code> <code>.foreach (a {!some}) { u a }</code> <code>.shell -ci "dir" cmd /c dir</code>

13. x86-to-x64 notes

The flow stays the same on x64, but the registers, pointer width, and a few structure offsets change. Treat this section as the delta list, not a second workflow.

Command	Why use it	Example
<code>r rcx / r rdx / r r8 / r r9</code> <code>dq addr</code> <code>dt ntdll!_PEB @\$peb</code> <code>dx @\$osed().sc.peb()</code> <code>dx @\$osed().sc.export("kernel32", "GetProcAddress")</code>	x64 first four arguments live in registers. Read 8-byte values on x64. The idea is the same even though the structure layout changes. The toolkit adapts to the active architecture. Export walking is still the same concept on x64.	<code>r rcx</code> <code>dq rsp L10</code> <code>dt ntdll!_PEB @\$peb</code> <code>dx @\$osed().sc.peb()</code> <code>dx @\$osed().sc.export("kernel32", "GetProcAddress")</code>

WoW64 note: for OSED-style 32-bit exploitation, use the 32-bit PEB and 32-bit register model. The x64 companion is for the native 64-bit side only. ARM and ARM64 notes live in [windbg-arm/README.md](#) as a separate delta sheet.

14. Suggested study order

1. Start the session and set symbols.
2. Read the crash state.
3. Inspect registers, stack, and memory.
4. Control execution and place breakpoints.
5. Search memory for patterns and bad characters.
6. Walk the PEB and loaded modules.
7. Read PE headers and exports.
8. Use the SEH, pivot, and gadget helpers.
9. Move to the x64 delta sheet once the x86 flow is comfortable.